

Task 16: Twenty CRM Failure & Recovery – Implementation Documentation

1. Project Objective

The objective of this task is to deploy Twenty CRM on an AWS EC2 instance using Docker Compose and configure automatic failure recovery at both the container and EC2 instance levels.

The recovery process should work when:

1. A Docker container crashes unexpectedly.
2. The EC2 instance is stopped and started again.

After an EC2 restart, the Docker service and containers should start automatically without manually running docker compose up.

2. Components

The application runs on an AWS EC2 instance using Docker Compose.

- AWS EC2 – Ubuntu Server 22.04 LTS.
- Instance type – t3.medium.
- Docker Engine – container runtime.
- Docker Compose – application orchestration.
- Twenty CRM – main CRM application.
- PostgreSQL and Redis – included in the Twenty CRM all-in-one server image.
- Docker volumes – persistent application data and storage.

3. AWS EC2 Configuration

The EC2 instance was provisioned through the AWS Console.

Configuration:

- Instance Name: myserver-fiza
- AML: Ubuntu Server 22.04 LTS
- Instance Type: t3.medium
- vCPU: 2
- RAM: 4 GB
- Storage: 30 GB gp3
- Application Port: 2020
- SSH Port: 22

Security Group:

- SSH (22) – restricted to the administrator IP.
- Custom TCP (2020) – 0.0.0.0/0 for application access.

For production environments, application access should be restricted to trusted networks or controlled through a suitable load balancer and security configuration.

4. Repository Structure

```
devops-crm-project/  
├── Dockerfile  
├── docker-compose.yml  
├── README.md  
└── fiza-task16.pdf
```

The Dockerfile builds the custom application image, while docker-compose.yml defines the services, volumes, healthcheck, and restart policies.

5. Prerequisites

- AWS account.
- Ubuntu Server 22.04 LTS EC2 instance.
- SSH key pair.
- Git.
- Docker Engine.
- Docker Compose plugin.
- Inbound TCP port 2020.
- Sufficient EC2 resources for the application.

6. Connecting to EC2

Connect to the instance using SSH:

```
ssh -i "twenty-crm-key.pem" ubuntu@<EC2_PUBLIC_IP>
```

Replace <EC2_PUBLIC_IP> with the public IPv4 address of the instance.

7. Installing Docker and Docker Compose

```
sudo apt update && sudo apt upgrade -y  
sudo apt install -y ca-certificates curl gnupg git  
  
sudo install -m 0755 -d /etc/apt/keyrings  
  
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \  
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg  
  
sudo chmod a+r /etc/apt/keyrings/docker.gpg  
  
echo \  
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]  
https://download.docker.com/linux/ubuntu \  
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \  
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null  
  
sudo apt update  
  
sudo apt install -y \  
docker-ce  
docker-ce-cli  
containerd  
docker-compose
```

```
docker-ce \
docker-ce-cli \
containerd.io \
docker-buildx-plugin \
docker-compose-plugin

sudo usermod -aG docker $USER
newgrp docker

sudo systemctl enable docker
sudo systemctl start docker
```

Verify the installation:

```
docker --version
docker compose version
sudo systemctl status docker --no-pager
```

Enabling Docker with `systemctl enable docker` is important because Docker must start automatically after an EC2 boot.

8. Cloning the Repository

```
git clone https://github.com/PearlThoughts-Intern-DevOps/devops-crm-project
cd devops-crm-project
git checkout fiza-task5
```

Make sure the branch contains the implementation being deployed.

9. Docker Compose Configuration

The application uses two services. The server service runs the official Twenty CRM all-in-one image. The app service is a custom image built from the repository Dockerfile.

The important configuration is:

```
services:
  server:
    image: twentycrm/twenty-app-dev:latest
    container_name: twenty-app-dev-server
    restart: unless-stopped
    ports:
      - "2020:2020"
    volumes:
      - twenty-app-dev-data:/app/data
      - twenty-app-dev-storage:/app/storage
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:2020/healthz || exit 1"]
      interval: 15s
      timeout: 10s
      retries: 30
      start_period: 300s

  app:
    image: devops-crm-app:local
    build:
      context: .
      dockerfile: Dockerfile
```

```
    container_name: devops-crm-app
    restart: unless-stopped
    depends_on:
      server:
        condition: service_healthy
    environment:
      TWENTY_SERVER_URL: http://server:2020

volumes:
  twenty-app-dev-data:
  twenty-app-dev-storage:
```

Restart Policy:

Both services use `restart: unless-stopped`. This allows Docker to restart containers after an unexpected failure and after the Docker daemon starts again, such as during an EC2 reboot. A container deliberately stopped by an administrator is not automatically restarted.

Healthcheck:

The server healthcheck checks the `/healthz` endpoint every 15 seconds. The 300-second `start_period` allows the all-in-one image enough time to initialize PostgreSQL, Redis, and the Twenty CRM application during startup.

Persistent Volumes:

The named volumes `twenty-app-dev-data` and `twenty-app-dev-storage` preserve application data and storage when containers are restarted or recreated.

10. Build and Start the Application

Build the custom application image:

```
docker compose build
```

Start the services:

```
docker compose up -d
```

Check the service status:

```
docker compose ps
```

Check all containers:

```
docker ps -a
```

View server logs:

```
docker compose logs server --tail=150
```

Wait until the server reports healthy and the app service has started.

11. Accessing Twenty CRM

Open the following URL in a browser:

```
http://<EC2_PUBLIC_IP>:2020
```

The Twenty CRM login or onboarding screen should appear. This confirms that the application is running and reachable.

12. Failure Recovery Configuration

Automatic recovery depends on three main configurations:

3. Docker restart policy – restart: unless-stopped.
4. Docker service auto-start – `sudo systemctl enable docker`.
5. Server healthcheck – verifies that Twenty CRM is healthy before the app service starts.

Together, these settings allow the application to recover from container failures and from an EC2 stop/start cycle.

13. Test 1 – Container Failure and Recovery

Objective: Verify that the Twenty CRM server container automatically restarts after an unexpected process failure.

Get the server container process ID:

```
CPID=$(docker inspect --format '{{.State.Pid}}' twenty-app-dev-server)
```

Terminate the process directly:

```
sudo kill -9 $CPID
```

This simulates an unexpected process failure. The process is terminated outside Docker's normal stop commands, allowing the restart policy to be tested.

Wait and verify:

```
sleep 10
docker ps -a

docker inspect twenty-app-dev-server \
  --format='status={{.State.Status}} RestartCount={{.RestartCount}}
  StartedAt={{.State.StartedAt}}'
```

Expected result:

- The container exits unexpectedly.
- Docker detects the failure.
- The container starts again automatically.
- RestartCount increases.
- StartedAt shows a new startup time.
- The container returns to running.
- The healthcheck eventually reports healthy.

14. Test 2 – EC2 Instance Stop/Start Recovery

Objective: Verify that Twenty CRM recovers automatically after the complete EC2 instance is stopped and started.

Procedure:

6. Open the AWS EC2 Console.
7. Select the myserver-fiza instance.
8. Choose Instance state → Stop instance.
9. Wait until the instance state becomes Stopped.
10. Choose Instance state → Start instance.
11. Wait until the instance state becomes Running.
12. Confirm that the EC2 status checks have passed.
13. Note the new public IPv4 address if it changed.
14. Reconnect to the instance using SSH.

Verify Docker:

```
sudo systemctl status docker --no-pager
```

Verify the containers:

```
docker ps -a
docker compose ps
```

Verify the application:

```
http://<NEW_EC2_PUBLIC_IP>:2020
```

Expected result:

- Docker starts automatically during EC2 boot.
- The server and app containers start automatically.
- The server becomes healthy.
- The application becomes reachable again.
- No manual docker compose up is required after the EC2 restart.

If the EC2 instance does not have an Elastic IP, the public IPv4 address can change after stopping and starting the instance.

15. Log Verification

Review the Twenty CRM server logs:

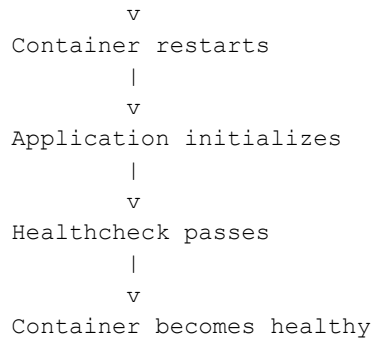
```
docker compose logs server --tail=150
```

Review Docker events:

```
docker events --since 1h \
  --filter container=twenty-app-dev-server \
  --filter event=die \
  --filter event=start \
  --filter event=health_status
```

The expected recovery sequence is:

```
Container process exits
|
v
Docker detects failure
|
```



16. Conclusion

Twenty CRM was deployed on AWS EC2 using Docker Compose with automatic failure recovery.

The implementation demonstrates recovery at two levels:

15. Container-level recovery – Docker automatically restarts the server container after an unexpected process failure.
16. Instance-level recovery – Docker starts automatically after EC2 boot, and the containers recover without manually running docker compose up.

The project demonstrates the use of Docker restart policies, healthchecks, persistent storage, and Docker service auto-start to improve application availability and recovery.